

Vigilant Matrix SDK Bridge

WebAssembly API Reference and Frontend Integration Guide

Vigilant Engineering

July 2026

API Reference and Frontend Integration Guide

Contents

1	Document Overview	5
1.1	Purpose	5
1.2	Scope	5
2	Architecture Overview	7
2.1	System Architecture	7
2.2	Runtime Homeserver Selection	7
2.3	Generated Package	7
3	Frontend Integration	9
3.1	Import and Initialization	9
4	API Lifecycle	10
5	Authentication API	11
5.1	MatrixBridge.init	11
5.1.1	Signature	11
5.1.2	Purpose	11
5.1.3	Parameters	11
5.1.4	Example	11
5.1.5	Notes	11
5.2	register	11
5.2.1	Signature	11
5.2.2	Purpose	11
5.2.3	Example	11
5.2.4	Cycle 1 Note	12
5.3	login	12
5.3.1	Signature	12
5.3.2	Purpose	12
5.3.3	Example	12
5.3.4	Errors	12
5.4	logout	12
5.4.1	Signature	12
5.4.2	Purpose	12
5.4.3	Example	12
6	Session Persistence	13
6.1	export_session	13
6.1.1	Signature	13
6.1.2	Purpose	13
6.1.3	Example	13
6.1.4	Security	13
6.2	restore_session	13
6.2.1	Signature	13
6.2.2	Purpose	13
6.2.3	Example	13
6.2.4	Typical Startup	14
7	Synchronization	15
7.1	start_sync	15
7.1.1	Signature	15
7.1.2	Purpose	15

7.1.3	Recommended Sequence	15
7.1.4	Important	15
7.2	<code>stop_sync</code>	15
7.2.1	Signature	15
7.2.2	Purpose	15
7.2.3	Example	15
8	Room Management	16
8.1	<code>create_room</code>	16
8.1.1	Signature	16
8.1.2	Purpose	16
8.1.3	Example	16
8.2	<code>list_joined_rooms</code>	16
8.2.1	Signature	16
8.2.2	Purpose	16
8.2.3	Return Value	16
8.2.4	Room Structure	16
8.3	<code>join_room</code>	17
8.3.1	Signature	17
8.3.2	Purpose	17
8.3.3	Example	17
8.4	<code>leave_room</code>	17
8.4.1	Signature	17
8.4.2	Purpose	17
8.4.3	Example	17
8.5	<code>invite_user</code>	17
8.5.1	Signature	17
8.5.2	Purpose	17
8.5.3	Example	17
9	Direct Messages	19
9.1	<code>create_direct_message</code>	19
9.1.1	Signature	19
9.1.2	Purpose	19
9.1.3	Example	19
9.2	<code>find_direct_message</code>	19
9.2.1	Signature	19
9.2.2	Purpose	19
9.2.3	Example	19
9.3	<code>get_or_create_direct_message</code>	19
9.3.1	Signature	20
9.3.2	Purpose	20
9.3.3	Recommended Usage	20
9.4	<code>list_direct_messages</code>	20
9.4.1	Signature	20
9.4.2	Purpose	20
9.4.3	Example	20
9.4.4	Structure	20
10	Text Messaging	21
10.1	<code>send_message</code>	21
10.1.1	Signature	21
10.1.2	Purpose	21
10.1.3	Example	21

11 Real-Time Messages	22
11.1 on_message	22
11.1.1 Signature	22
11.1.2 Purpose	22
11.1.3 Example	22
11.1.4 JsMessage	22
11.1.5 Text Example	22
11.1.6 Supported Message Types	23
12 Notifications	24
12.1 on_notification	24
12.1.1 Signature	24
12.1.2 Purpose	24
12.1.3 Example	24
12.1.4 Notification Structure	24
13 File and Image Messaging	25
13.1 send_image	25
13.1.1 Signature	25
13.1.2 Purpose	25
13.1.3 Example	25
13.2 send_file	25
13.2.1 Signature	25
13.2.2 Purpose	25
13.2.3 Cycle 1 Restriction	25
13.2.4 Example	26
14 Media Retrieval	27
14.1 get_media	27
14.1.1 Signature	27
14.1.2 Purpose	27
14.1.3 Example	27
15 Message History	28
15.1 get_room_history	28
15.1.1 Signature	28
15.1.2 Purpose	28
15.1.3 Return Value	28
15.1.4 Example	28
15.2 load_more_history	28
15.2.1 Signature	28
15.2.2 Purpose	28
15.2.3 Example	29
15.2.4 Requirement	29
16 History Behavior in Cycle 1	30
17 Recommended Conversation Integration	31
18 Recommended Application Startup	33
19 Data Contracts	34
19.1 JsRoom	34
19.2 JsDirectMessage	34
19.3 JsMessage	34

19.4 Notification	34
19.5 HistoryResponse	34
20 Error Handling	36
20.1 Matrix Error Examples	36
20.1.1 M_UNKNOWN_TOKEN	36
20.1.2 M_LIMIT_EXCEEDED	36
20.1.3 M_USER_IN_USE	36
20.1.4 M_FORBIDDEN	36
21 Frontend Rules	37
22 Known Cycle 1 Limitations	38
23 API Summary	39
24 Appendix A: Matrix Identifier Reference	40
24.1 User ID	40
24.2 Room ID	40
24.3 MXC Media URI	40
25 Appendix B: Minimal Working Integration	41
26 Appendix C: Backend Cycle 1 Baseline	42
27 Appendix D: Build and Handoff	44
28 Conclusion	45

1 Document Overview

1.1 Purpose

This document specifies the public frontend API of the **Vigilant Matrix SDK Bridge**, a Rust-based WebAssembly interface built over the Matrix Rust SDK.

The bridge provides a simplified JavaScript-facing API for Matrix functionality required by the Vigilant frontend. It hides the Rust and Matrix SDK implementation details behind a comparatively small set of asynchronous JavaScript methods.

This document is intended primarily for frontend integration. It defines:

- bridge initialization and homeserver configuration;
- authentication and session management;
- Matrix synchronization;
- room management;
- direct-message management;
- real-time messaging;
- notifications;
- message history and pagination;
- image and PDF transfer;
- media retrieval;
- returned JSON structures;
- expected frontend workflows;
- errors and Cycle 1 limitations.

The API described here corresponds to **Backend Cycle 1, version 0.1**.

1.2 Scope

Cycle 1 establishes a functional Matrix communication layer suitable for integration with the Vigilant frontend.

The current bridge supports:

Capability	Status
Runtime homeserver selection	Supported
User registration	Supported
Login and logout	Supported
Session export and restoration	Supported
Matrix synchronization	Supported
Room creation, joining and leaving	Supported
User invitation	Supported
Direct messages	Supported
Text messaging	Supported
Real-time message callbacks	Supported
Notification callbacks	Supported
Image upload and download	Supported
PDF upload and download	Supported
Room message history	Supported
Backward history pagination	Supported
End-to-end production authentication workflow	Deferred
Production deployment hardening	Deferred

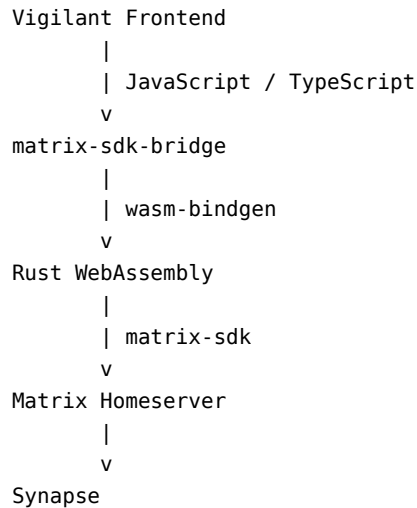
Features marked as deferred are intentionally outside the scope of Backend Cycle 1.

2 Architecture Overview

2.1 System Architecture

The frontend does not communicate with the Matrix Rust SDK directly.

The communication path is:



The bridge therefore acts as an abstraction boundary between the frontend and the underlying Matrix implementation.

The frontend should depend on the public `MatrixBridge` interface documented here rather than Matrix SDK internals.

2.2 Runtime Homeserver Selection

The homeserver is supplied when the bridge is initialized.

Development:

```
const bridge = await MatrixBridge.init(
  "http://localhost:8008"
);
```

Tunnel/deployed environment:

```
const bridge = await MatrixBridge.init(
  "https://matrix.seucra.tech"
);
```

This means the same compiled WebAssembly package can target different Matrix homeservers without recompiling the Rust project.

2.3 Generated Package

Running:

```
wasm-pack build --target web
```

generates the browser-consumable package under `pkg/`.

Important generated artifacts include:

```
pkg/  
├─ matrix_sdk_bridge.js  
├─ matrix_sdk_bridge_bg.wasm  
├─ matrix_sdk_bridge.d.ts  
├─ matrix_sdk_bridge_bg.wasm.d.ts  
└─ package.json
```

Frontend code should interact with the generated JavaScript API rather than importing or depending on Rust source modules.

3 Frontend Integration

3.1 Import and Initialization

A basic browser integration begins by loading the generated WASM package.

```
import init, { MatrixBridge }  
  from "../pkg/matrix_sdk_bridge.js";  
  
await init();
```

```
const bridge = await MatrixBridge.init(  
  "http://localhost:8008"  
);
```

For a deployed frontend:

```
const bridge = await MatrixBridge.init(  
  "https://matrix.seucra.tech"  
);
```

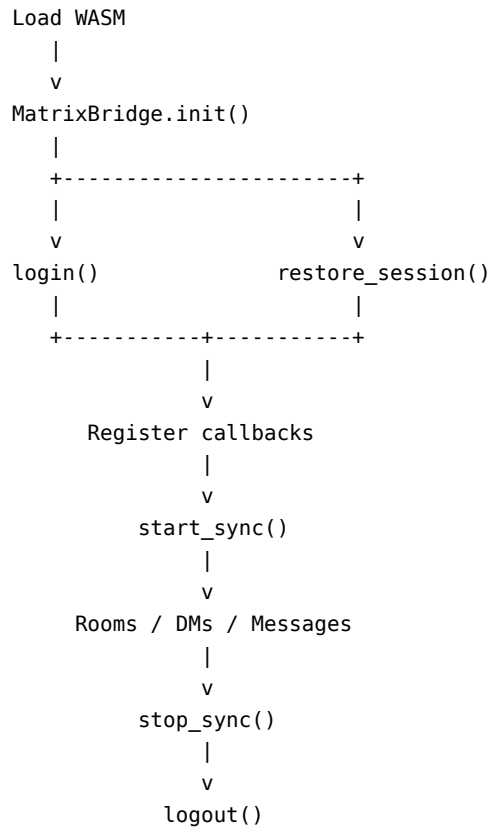
The homeserver URL should normally be stored in frontend configuration rather than repeated throughout application code.

For example:

```
const HOMESERVER =  
  "https://matrix.seucra.tech";  
  
const bridge =  
  await MatrixBridge.init(HOMESERVER);
```

4 API Lifecycle

A normal application lifecycle is approximately:



Callbacks should normally be registered before synchronization begins.

5 Authentication API

5.1 MatrixBridge.init

5.1.1 Signature

```
static init(  
    homeserver_url: string  
): Promise<MatrixBridge>;
```

5.1.2 Purpose

Creates a new Matrix bridge instance configured for the specified homeserver.

5.1.3 Parameters

Parameter	Type	Description
homeserver_url	string	Base URL of the Matrix homeserver

5.1.4 Example

```
const bridge = await MatrixBridge.init(  
    "https://matrix.seucra.tech"  
);
```

5.1.5 Notes

Calling init() creates the bridge but does not authenticate the user.

5.2 register

5.2.1 Signature

```
register(  
    username: string,  
    password: string  
): Promise<string>;
```

5.2.2 Purpose

Registers a new Matrix account.

5.2.3 Example

```
try {  
    const result =  
        await bridge.register("alice", password);  
  
    console.log(result);  
} catch (error) {  
    console.error("Registration failed:", error);  
}
```

5.2.4 Cycle 1 Note

The Cycle 1 Synapse environment uses a simplified registration configuration. Production-grade verification, CAPTCHA and related registration controls are outside the current cycle.

5.3 login

5.3.1 Signature

```
login(  
  username: string,  
  password: string  
): Promise<string>;
```

5.3.2 Purpose

Authenticates the bridge as an existing Matrix user.

5.3.3 Example

```
const result =  
  await bridge.login("alice", password);  
  
console.log(result);
```

A successful login returns a human-readable status string.

5.3.4 Errors

Possible Matrix errors include authentication failure and homeserver rate limiting.

For example, repeated login attempts may produce:

M_LIMIT_EXCEEDED

The frontend should not repeatedly retry authentication without respecting server rate limits.

5.4 logout

5.4.1 Signature

```
logout(): Promise<string>;
```

5.4.2 Purpose

Logs out the currently authenticated Matrix session.

5.4.3 Example

```
await bridge.stop_sync();  
const result = await bridge.logout();
```

The frontend should also remove any locally persisted exported session after a successful logout.

6 Session Persistence

6.1 export_session

6.1.1 Signature

```
export_session(): string | undefined;
```

6.1.2 Purpose

Exports the current authenticated Matrix session into a JSON string suitable for local persistence.

6.1.3 Example

```
const session = bridge.export_session();

if (session) {
  localStorage.setItem(
    "matrix_session",
    session
  );
}
```

6.1.4 Security

The exported session contains authentication material.

It must be treated as sensitive application data.

It should never be:

- printed into production logs;
 - committed to source control;
 - embedded into frontend source code;
 - transmitted to unrelated services.
-

6.2 restore_session

6.2.1 Signature

```
restore_session(
  session_json: string
): Promise<string>;
```

6.2.2 Purpose

Restores a previously exported Matrix session.

6.2.3 Example

```
const saved =
  localStorage.getItem("matrix_session");

if (saved) {
  await bridge.restore_session(saved);
}
```

```
}
```

6.2.4 Typical Startup

```
await init();

const bridge =
  await MatrixBridge.init(HOMESERVER);

const saved =
  localStorage.getItem("matrix_session");

if (saved) {
  await bridge.restore_session(saved);
} else {
  await bridge.login(username, password);
}
```

7 Synchronization

Synchronization is essential to Matrix operation.

Authentication establishes identity. Synchronization populates and continuously updates Matrix state.

Without synchronization, room membership and incoming events may not reflect the current homeserver state.

7.1 `start_sync`

7.1.1 Signature

```
start_sync(): void;
```

7.1.2 Purpose

Starts Matrix synchronization.

The bridge performs an initial synchronization and subsequently continues synchronization in the background.

7.1.3 Recommended Sequence

```
bridge.on_message(handleMessage);  
bridge.on_notification(handleNotification);
```

```
bridge.start_sync();
```

7.1.4 Important

Only one synchronization loop should run for a bridge instance.

Calling `start_sync()` again while synchronization is already active produces an error such as:

Sync is already running

7.2 `stop_sync`

7.2.1 Signature

```
stop_sync(): void;
```

7.2.2 Purpose

Requests termination of the active synchronization loop.

7.2.3 Example

```
bridge.stop_sync();
```


8 Room Management

8.1 create_room

8.1.1 Signature

```
create_room(  
  name: string  
): Promise<string>;
```

8.1.2 Purpose

Creates a Matrix room.

8.1.3 Example

```
const roomId =  
  await bridge.create_room("Vigilant Test");  
  
console.log(roomId);
```

A Matrix room ID resembles:

```
!rIxQkLvUwQKITMuKPS:matrix.seucra.tech
```

8.2 list_joined_rooms

8.2.1 Signature

```
list_joined_rooms(): Promise<string>;
```

8.2.2 Purpose

Returns the rooms currently joined by the authenticated user.

8.2.3 Return Value

The method returns a JSON-encoded string.

Frontend code should parse it:

```
const raw =  
  await bridge.list_joined_rooms();  
  
const rooms = JSON.parse(raw);
```

8.2.4 Room Structure

A room follows the conceptual structure:

```
{  
  "room_id": "!example:matrix.seucra.tech",  
  "name": "Vigilant Test"  
}
```

8.3 join_room

8.3.1 Signature

```
join_room(  
    room_id_or_alias: string  
): Promise<string>;
```

8.3.2 Purpose

Joins a Matrix room using its room ID or supported alias.

8.3.3 Example

```
await bridge.join_room(  
    "!example:matrix.seucra.tech"  
);
```

8.4 leave_room

8.4.1 Signature

```
leave_room(  
    room_id_str: string  
): Promise<string>;
```

8.4.2 Purpose

Leaves a currently joined room.

8.4.3 Example

```
await bridge.leave_room(roomId);
```

8.5 invite_user

8.5.1 Signature

```
invite_user(  
    room_id_str: string,  
    user_id_str: string  
): Promise<string>;
```

8.5.2 Purpose

Invites another Matrix user to a room.

8.5.3 Example

```
await bridge.invite_user(  
    roomId,  
    "@bob:matrix.seucra.tech"  
);
```

Matrix user IDs must use their complete Matrix identifier.

9 Direct Messages

The bridge provides higher-level direct-message operations so the frontend does not need to manually reproduce Matrix DM bookkeeping.

9.1 create_direct_message

9.1.1 Signature

```
create_direct_message(  
  user_id_str: string  
): Promise<string>;
```

9.1.2 Purpose

Creates a direct-message room for another Matrix user.

9.1.3 Example

```
const roomId =  
  await bridge.create_direct_message(  
    "@bob:matrix.seucra.tech"  
  );
```

9.2 find_direct_message

9.2.1 Signature

```
find_direct_message(  
  user_id_str: string  
): Promise<string | undefined>;
```

9.2.2 Purpose

Searches existing direct-message metadata for a DM associated with the specified user.

9.2.3 Example

```
const roomId =  
  await bridge.find_direct_message(  
    "@bob:matrix.seucra.tech"  
  );  
  
if (roomId) {  
  console.log("Existing DM:", roomId);  
}
```

9.3 get_or_create_direct_message

9.3.1 Signature

```
get_or_create_direct_message(
  user_id_str: string
): Promise<string>;
```

9.3.2 Purpose

Convenience operation intended for normal frontend DM creation.

It attempts to reuse an existing direct-message room and creates one when necessary.

9.3.3 Recommended Usage

For a normal “Message User” button:

```
const roomId =
  await bridge.get_or_create_direct_message(
    targetUserId
  );
```

```
openConversation(roomId);
```

This should generally be preferred over calling `create_direct_message()` directly.

9.4 list_direct_messages

9.4.1 Signature

```
list_direct_messages(): Promise<string>;
```

9.4.2 Purpose

Returns known direct-message rooms as JSON.

9.4.3 Example

```
const raw =
  await bridge.list_direct_messages();
```

```
const directMessages =
  JSON.parse(raw);
```

9.4.4 Structure

```
[
  {
    "room_id":
      "!example:matrix.seucra.tech",
    "targets": [
      "@bob:matrix.seucra.tech"
    ],
    "name": "bob"
  }
]
```

10 Text Messaging

10.1 `send_message`

10.1.1 Signature

```
send_message(  
    room_id_str: string,  
    message: string  
): Promise<string>;
```

10.1.2 Purpose

Sends a plain-text Matrix message.

10.1.3 Example

```
await bridge.send_message(  
    roomId,  
    "Hello from Vigilant"  
);
```

The room must exist and be accessible to the authenticated user.

11 Real-Time Messages

11.1 on_message

11.1.1 Signature

```
on_message(  
  callback: Function  
): void;
```

11.1.2 Purpose

Registers a JavaScript callback for supported Matrix message events.

The callback receives a JSON-encoded `JsMessage`.

11.1.3 Example

```
bridge.on_message((json) => {  
  const message = JSON.parse(json);  
  
  console.log(  
    message.sender,  
    message.body  
  );  
});
```

11.1.4 JsMessage

```
interface JsMessage {  
  room_id: string;  
  sender: string;  
  body: string;  
  timestamp: number;  
  
  message_type: string;  
  message_uri: string | null;  
  mime_type: string | null;  
  media_source: string | null;  
}
```

11.1.5 Text Example

```
{  
  "room_id":  
    "!example:matrix.seucra.tech",  
  "sender":  
    "@alice:matrix.seucra.tech",  
  "body": "Hello",  
  "timestamp": 1785015727686,  
  "message_type": "text",  
  "message_uri": null,  
  "mime_type": null,  
  "media_source": null  
}
```

11.1.6 Supported Message Types

Cycle 1 currently extracts:

text
image
file

Unsupported Matrix message types are not exposed through this simplified message abstraction.

12 Notifications

12.1 on_notification

12.1.1 Signature

```
on_notification(  
  callback: Function  
): void;
```

12.1.2 Purpose

Registers a callback for notification-like events exposed by the bridge.

12.1.3 Example

```
bridge.on_notification((json) => {  
  const notification =  
    JSON.parse(json);  
  
  console.log(notification);  
});
```

12.1.4 Notification Structure

```
interface Notification {  
  event_type: string;  
  room_id: string;  
  sender: string;  
  body: string;  
}
```

Example:

```
{  
  "event_type": "text",  
  "room_id":  
    "!example:matrix.seucra.tech",  
  "sender":  
    "@bob:matrix.seucra.tech",  
  "body": "Hello"  
}
```

13 File and Image Messaging

13.1 send_image

13.1.1 Signature

```
send_image(  
  room_id_str: string,  
  data: Uint8Array,  
  filename: string,  
  mime_type: string  
): Promise<string>;
```

13.1.2 Purpose

Uploads and sends an image to a Matrix room.

13.1.3 Example

```
const bytes =  
  new Uint8Array(  
    await file.arrayBuffer()  
  );  
  
await bridge.send_image(  
  roomId,  
  bytes,  
  file.name,  
  file.type  
);
```

The supplied MIME type must be an image MIME type.

13.2 send_file

13.2.1 Signature

```
send_file(  
  room_id_str: string,  
  data: Uint8Array,  
  filename: string,  
  mime_type: string  
): Promise<string>;
```

13.2.2 Purpose

Uploads and sends a supported file.

13.2.3 Cycle 1 Restriction

The current implementation intentionally accepts PDF files only.

13.2.4 Example

```
const bytes =  
  new Uint8Array(  
    await file.arrayBuffer()  
  );  
  
await bridge.send_file(  
  roomId,  
  bytes,  
  file.name,  
  "application/pdf"  
);
```

Attempting to send another MIME type through `send_file()` results in an error.

14 Media Retrieval

14.1 get_media

14.1.1 Signature

```
get_media(  
  media_source_json: string  
): Promise<Uint8Array>;
```

14.1.2 Purpose

Retrieves media associated with a received image or file message.

`media_source_json` should come from the `media_source` field returned in `JsMessage`.

The frontend should treat this value as opaque bridge data.

It should not attempt to construct Matrix media-source JSON manually.

14.1.3 Example

```
const bytes =  
  await bridge.get_media(  
    message.media_source  
  );  
  
const blob =  
  new Blob(  
    [bytes],  
    { type: message.mime_type }  
  );  
  
const url =  
  URL.createObjectURL(blob);
```

For an image:

```
imageElement.src = url;
```

For a downloadable PDF:

```
const anchor =  
  document.createElement("a");  
  
anchor.href = url;  
anchor.download = message.body;  
anchor.click();
```

The frontend should eventually release generated object URLs:

```
URL.revokeObjectURL(url);
```

15 Message History

15.1 get_room_history

15.1.1 Signature

```
get_room_history(  
  room_id_str: string,  
  limit: number  
): Promise<string>;
```

15.1.2 Purpose

Initializes or accesses the timeline for a room and requests older history.

15.1.3 Return Value

A JSON-encoded HistoryResponse.

```
interface HistoryResponse {  
  messages: JsMessage[];  
  has_more: boolean;  
}
```

15.1.4 Example

```
const raw =  
  await bridge.get_room_history(  
    roomId,  
    50  
  );  
  
const history =  
  JSON.parse(raw);  
  
renderMessages(history.messages);  
  
if (history.has_more) {  
  enableLoadMore();  
}
```

15.2 load_more_history

15.2.1 Signature

```
load_more_history(  
  room_id_str: string,  
  limit: number  
): Promise<string>;
```

15.2.2 Purpose

Requests additional older events for a timeline previously initialized through get_room_history().

15.2.3 Example

```
const raw =
  await bridge.load_more_history(
    roomId,
    50
  );

const history =
  JSON.parse(raw);

prependMessages(history.messages);
```

15.2.4 Requirement

`get_room_history()` must initialize the room timeline before `load_more_history()` is used.

Otherwise an error similar to the following may occur:

Timeline not initialized

16 History Behavior in Cycle 1

Timeline loading is asynchronous and depends on Matrix SDK timeline state.

During testing, the first history request may occasionally return:

```
{  
  "messages": [],  
  "has_more": true  
}
```

while a subsequent pagination/history operation populates the timeline and returns the expected historical messages.

This is a known **Cycle 1 integration behavior**, not evidence that room history has been lost.

Frontend code should therefore avoid assuming that one empty initial response necessarily means that a room contains no messages.

This behavior should be revisited during a later backend cycle when timeline initialization and frontend loading states are refined.

17 Recommended Conversation Integration

A practical frontend conversation screen can use the bridge as follows:

```
async function openRoom(roomId) {
  const raw =
    await bridge.get_room_history(
      roomId,
      50
    );

  const history =
    JSON.parse(raw);

  renderMessages(history.messages);
}
```

Real-time messages are handled independently:

```
bridge.on_message((json) => {
  const message = JSON.parse(json);

  if (
    message.room_id === currentRoomId
  ) {
    appendMessage(message);
  }
});
```

Sending:

```
async function sendText(text) {
  await bridge.send_message(
    currentRoomId,
    text
  );
}
```

Loading older messages:

```
async function loadOlder() {
  const raw =
    await bridge.load_more_history(
      currentRoomId,
      50
    );

  const result =
    JSON.parse(raw);

  prependMessages(result.messages);
}
```

This separation is important:

```
get_room_history()
|
+---- historical state
```



```
on_message()  
  |  
  +---- real-time state
```

The frontend combines both into its displayed conversation.

18 Recommended Application Startup

A complete simplified startup sequence is:

```
import init, { MatrixBridge }
  from "../pkg/matrix_sdk_bridge.js";

const HOMESERVER =
  "https://matrix.seucra.tech";

await init();

const bridge =
  await MatrixBridge.init(HOMESERVER);

const savedSession =
  localStorage.getItem(
    "matrix_session"
  );

if (savedSession) {
  await bridge.restore_session(
    savedSession
  );
} else {
  await bridge.login(
    username,
    password
  );

  const exported =
    bridge.export_session();

  if (exported) {
    localStorage.setItem(
      "matrix_session",
      exported
    );
  }
}

bridge.on_message((json) => {
  const message = JSON.parse(json);
  handleIncomingMessage(message);
});

bridge.on_notification((json) => {
  const notification =
    JSON.parse(json);

  handleNotification(notification);
});

bridge.start_sync();
```

19 Data Contracts

19.1 JsRoom

```
interface JsRoom {  
  room_id: string;  
  name: string;  
}
```

19.2 JsDirectMessage

```
interface JsDirectMessage {  
  room_id: string;  
  targets: string[];  
  name: string;  
}
```

19.3 JsMessage

```
interface JsMessage {  
  room_id: string;  
  sender: string;  
  body: string;  
  timestamp: number;  
  
  message_type:  
    "text" | "image" | "file";  
  
  message_uri:  
    string | null;  
  
  mime_type:  
    string | null;  
  
  media_source:  
    string | null;  
}
```

19.4 Notification

```
interface Notification {  
  event_type: string;  
  room_id: string;  
  sender: string;  
  body: string;  
}
```

19.5 HistoryResponse

```
interface HistoryResponse {  
  messages: JsMessage[];
```

```
    has_more: boolean;  
}
```

20 Error Handling

Most asynchronous bridge methods reject their returned Promise when the underlying Rust or Matrix operation fails.

Frontend calls should therefore use either `try/catch`:

```
try {
  await bridge.send_message(
    roomId,
    text
  );
} catch (error) {
  console.error(
    "Message failed:",
    error
  );
}
```

or Promise rejection handling.

Errors can originate from:

- invalid Matrix identifiers;
- authentication failure;
- expired or invalid access tokens;
- rate limiting;
- unavailable rooms;
- insufficient room permissions;
- invalid MIME types;
- unavailable media;
- network failure;
- homeserver failure;
- incorrect API lifecycle usage.

20.1 Matrix Error Examples

20.1.1 M_UNKNOWN_TOKEN

The stored access token is no longer accepted.

The frontend should normally clear the persisted session and require authentication again.

20.1.2 M_LIMIT_EXCEEDED

The homeserver has rate-limited the request.

Repeated immediate retries should be avoided.

20.1.3 M_USER_IN_USE

Registration attempted to create a username that already exists.

20.1.4 M_FORBIDDEN

The requested Matrix operation is not permitted in the current state or with the current user's permissions.

21 Frontend Rules

The following integration rules should be treated as part of the API contract.

1. Initialize WASM before creating `MatrixBridge`.
2. Supply the homeserver URL to `MatrixBridge.init()`.
3. Authenticate using `login()` or `restore_session()`.
4. Register message and notification callbacks before starting normal synchronization.
5. Do not start multiple synchronization loops on the same bridge.
6. Treat Matrix room IDs and user IDs as opaque identifiers.
7. Parse methods documented as returning JSON strings using `JSON.parse()`.
8. Treat `media_source` as opaque bridge data and pass it back to `get_media()` unchanged.
9. Convert browser files to `Uint8Array` before sending them to the bridge.
10. Do not expose exported Matrix session data.
11. Initialize room history before requesting additional history.
12. Expect Matrix operations to be asynchronous and potentially network-dependent.

22 Known Cycle 1 Limitations

Backend Cycle 1 intentionally prioritizes functional vertical integration over production completeness.

Known limitations include:

- simplified registration configuration;
- limited exposed Matrix message types;
- `send_file()` restricted to PDF;
- timeline initialization may require an additional pagination/history request before historical messages appear;
- notification behavior is intentionally minimal;
- frontend-friendly typed JavaScript objects are not yet returned directly for all APIs, so several methods return JSON strings;
- production authentication and account-recovery workflows are not implemented;
- deployment and infrastructure hardening remain outside this API cycle.

These limitations should be treated as candidates for subsequent spiral-development cycles rather than hidden defects.

23 API Summary

Method	Purpose	Return
<code>init(url)</code>	Create bridge	<code>MatrixBridge</code>
<code>register(user, pass)</code>	Register user	<code>Promise<string></code>
<code>login(user, pass)</code>	Authenticate	<code>Promise<string></code>
<code>logout()</code>	Logout	<code>Promise<string></code>
<code>export_session()</code>	Export session	<code>string</code> <code> undefined</code>
<code>restore_session(json)</code>	Restore session	<code>Promise<string></code>
<code>start_sync()</code>	Start synchronization	<code>void</code>
<code>stop_sync()</code>	Stop synchronization	<code>void</code>
<code>create_room(name)</code>	Create room	<code>Promise<string></code>
<code>list_joined_rooms()</code>	List rooms	<code>Promise<string></code>
<code>join_room(id)</code>	Join room	<code>Promise<string></code>
<code>leave_room(id)</code>	Leave room	<code>Promise<string></code>
<code>invite_user(room,user)</code>	Invite user	<code>Promise<string></code>
<code>create_direct_message(user)</code>	Create DM	<code>Promise<string></code>
<code>find_direct_message(user)</code>	Find DM	<code>Promise<string</code> <code> undefined></code>
<code>get_or_create_direct_message(user)</code>	Resolve DM	<code>Promise<string></code>
<code>list_direct_messages()</code>	List DMs	<code>Promise<string></code>
<code>send_message(room,text)</code>	Send text	<code>Promise<string></code>
<code>on_message(callback)</code>	Receive messages	<code>void</code>
<code>on_notification(callback)</code>	Receive notifications	<code>void</code>
<code>send_image(...)</code>	Send image	<code>Promise<string></code>
<code>send_file(...)</code>	Send PDF	<code>Promise<string></code>
<code>get_media(source)</code>	Retrieve media	<code>Promise<Uint8Array></code>
<code>get_room_history(room,limit)</code>	Get history	<code>Promise<string></code>
<code>load_more_history(room,limit)</code>	Paginate history	<code>Promise<string></code>

24 Appendix A: Matrix Identifier Reference

24.1 User ID

`@alice:matrix.seucra.tech`

General form:

`@localpart:server`

24.2 Room ID

`!rIxQkLvUwQKITMuKPS:matrix.seucra.tech`

Room IDs should be stored and passed unchanged.

24.3 MXC Media URI

Matrix media may internally reference an MXC URI.

The frontend generally does not need to resolve MXC resources itself because `get_media()` handles retrieval through the Matrix SDK.

25 Appendix B: Minimal Working Integration

```
import init, { MatrixBridge }
  from "../pkg/matrix_sdk_bridge.js";

await init();

const bridge =
  await MatrixBridge.init(
    "https://matrix.seucra.tech"
  );

await bridge.login(
  "alice",
  password
);

bridge.on_message((json) => {
  const message = JSON.parse(json);

  console.log(
    `[${message.room_id}]`,
    message.sender,
    message.body
  );
});

bridge.start_sync();

const rooms =
  JSON.parse(
    await bridge.list_joined_rooms()
  );

console.log(rooms);

if (rooms.length > 0) {
  await bridge.send_message(
    rooms[0].room_id,
    "Vigilant bridge operational."
  );
}
```

26 Appendix C: Backend Cycle 1 Baseline

The following functionality has been exercised during Cycle 1 integration testing:

WASM initialization

✓

User registration

✓

Login

✓

Session export / restore

✓

Room creation

✓

Room joining / leaving

✓

Room invitation

✓

Room discovery after sync

✓

Direct-message creation

✓

Direct-message discovery

✓

Real-time text messaging

✓

Message callbacks

✓

Notification callbacks

✓

PDF upload

✓

Image upload

✓

Media download

✓

Room history retrieval

✓

Backward pagination

✓

Runtime homeserver configuration

✓

This baseline defines the functional scope of **Vigilant Backend Cycle 1, version 0.1**.

27 Appendix D: Build and Handoff

Build the bridge using:

```
cargo check  
wasm-pack build --target web
```

The resulting `pkg/` directory provides the WebAssembly module and JavaScript/TypeScript bindings consumed by the frontend.

For Backend Cycle 1, the generated package may be committed alongside the source to provide a fixed integration artifact to the frontend developer.

A future development cycle may replace this handoff mechanism with package publication and conventional dependency management.

28 Conclusion

The Vigilant Matrix SDK Bridge provides a WebAssembly abstraction over the Matrix Rust SDK intended to keep Matrix-specific complexity outside the frontend application.

Backend Cycle 1 establishes the core communication path from browser JavaScript through WebAssembly to the configured Matrix homeserver, including authentication, session persistence, synchronization, rooms, direct messages, real-time messaging, media transfer and message history.

The frontend should treat the documented `MatrixBridge` interface and its serialized data structures as the integration boundary. Internal Rust and Matrix SDK implementation details are not part of the frontend contract.

Subsequent development cycles may extend or refine this contract while preserving compatibility where practical.